

CS3364 Algorithms

Project 1

Finding Source Reliability by Counting Inversions During Sorting

Quicksort, Merge Sort, and Insertion Sort

Report prepared by: Mia Isabella Troncoso

Date: October 2, 2026

Abstract

This project measures disagreement between source rankings and a combined ranking by counting inversions. Three sorting approaches are examined: stable-partition Quicksort, merge sort, and insertion sort. Each algorithm links sorting operations to pairs that appear in the wrong relative order. Experiments use five supplied sequences of 10,000 integers and independently verify their inversion counts.

The completed implementation uses stable-partition Quicksort, corrected merge sort, and insertion sort. The merge method adds the number of remaining left elements when a smaller right value is selected. All algorithms return long counts and preserve a sorted result. A repaired driver reads and validates input, invokes each algorithm on a fresh copy, checks sorting, and reports source reliability. Both the finished driver and an independent test harness were exercised.

The verified counts rank source2 as the most reliable and source4 as the least reliable under the assignment definition. Local timing measurements show insertion sort is slower on these datasets, while Quicksort and merge sort have similar millisecond-scale execution times. Merge sort offers guaranteed $O(n \log n)$ sorting time; fixed-pivot Quicksort has an $O(n^2)$ worst case.

Report status: Final report for the repaired implementation, including its design, verified experiments, integration checks, and remaining limits. The accompanying package contains the four Java source files and all five datasets.

1. Interpretation of the Given Problem

1.1 Ranking disagreement

A search engine may combine rankings from multiple sources and later give greater weight to sources that agree more closely with the combined result. The assignment defines reliability through ordering disagreement, rather than factual accuracy. A source with fewer inversions is more reliable according to this model.

For an array A of length n , an inversion is a pair of indices (i, j) satisfying $0 \leq i < j < n$ and $A[i] > A[j]$. The number of inversions is the count of all such pairs. Equal values do not form inversions. A nondecreasing sequence has zero inversions; a reverse-ordered sequence of distinct values has $n(n-1)/2$ inversions.

1.2 Example and input interpretation

The assignment example $[4, 5, 8, 9, 6]$ has exactly two inversions: $(8, 6)$ and $(9, 6)$. After sorting, the array becomes $[4, 5, 6, 8, 9]$. The desired result is the number of inversions in the original sequence, not the zero inversions in the finished sorted array.

The five files `source1.txt` through `source5.txt` each contain 10,000 integer values. The experiments treat their order as the page order induced by the same combined ranking. The files do not supply explicit page identifiers in the tested representation. Accordingly, interpreting counts as source reliability depends on this shared-order assumption. The code does not construct a combined ranking from page IDs.

1.3 Objectives and output

The project requires modifying merge sort, Quicksort, and a third chosen sorting algorithm to count inversions. Insertion sort is the third method. The output should include a sorted sequence, its original inversion count, and a comparison of the sources. Every algorithm must start from a fresh copy of the same original sequence; otherwise, an algorithm run after another sort would incorrectly report zero.

For 10,000 values, the maximum count is 49,995,000, which fits in a Java int. All completed algorithms use long counters because larger inputs can exceed the int limit. The independent reference also uses long.

1.4 Report organization

Sections 2-4 explain the algorithms and their correctness. Sections 5-6 describe correctness experiments and runtime measurements. Section 7 describes the repaired driver and integration checks. Section 8 presents conclusions, future work, team contributions, and the materials used.

2. Quicksort Methodology

2.1 Stable partition and recursive counting

The Quicksort implementation supplied earlier in this conversation selects the last value of a subarray as its pivot. It scans the preceding values and places each into a temporary small array if it is less than or equal to the pivot, or into a temporary large array if it is greater. Both groups preserve their original relative order. The array is rebuilt as small values, pivot, and large values, and the two groups are recursively sorted.

The variable `largerSeen` counts previously encountered values greater than the pivot. A large value adds one inversion with the final pivot. A small value adds `largerSeen` inversions because every previously seen large value is greater than it and originally occurs before it. Each call returns the partition count plus the counts from its two recursive calls.

```
if (array[i] > pivot) {
    large[largeCount++] = array[i];
    largerSeen++;
    cross += 1;
} else {
    small[smallCount++] = array[i];
    cross += largerSeen;
}
```

2.2 Correctness argument

Every inversion falls into exactly one category: both values in the small group; both in the large group; an earlier large value and a later small value; or a large value followed by the pivot. Recursive calls count the first two categories, and the partition scan counts the last two. Preserving order inside each group is essential because the recursive counts must reflect the original pair order. These categories are disjoint and exhaustive, so the final sum counts every inversion once.

2.3 Worked example

Input: [3, 1, 2]	Partition count
Pivot is 2. Reading 3 adds the pair (3, 2).	1
Reading 1 adds the earlier-large pair (3, 1).	2
Rebuild as [1, 2, 3]; recursive groups are singletons.	Final count: 2

2.4 Complexity and limitations

Partitioning costs $O(n)$ per call. Balanced partitions give $O(n \log n)$ time, and average behavior has that bound under suitable input-order assumptions. A last-element pivot can produce $O(n^2)$ time on sorted, reverse-sorted, or all-equal inputs. Recursion depth ranges from $O(\log n)$ to $O(n)$. Balanced auxiliary storage is $O(n)$; retaining temporary arrays down a maximally unbalanced path can require $O(n^2)$ storage. This implementation is not in place.

3. Merge-Sort Methodology

3.1 Divide, sort, and merge

Merge sort divides a sequence into two approximately equal parts, recursively sorts both, and merges the sorted results. The supplied code rounds the left-half size upward for odd lengths; this is valid. During merging, indices i and j identify the next available values in the left and right arrays. Selecting the left value when $L[i] \leq R[j]$ preserves stability and avoids counting equal values as inversions.

3.2 Submitted defect and required correction

The original attached merge method called `addInversion()`, which incremented `inv` by one whenever the right-side value was smaller. A single right-side value can be smaller than several remaining left-side values. The final `MergeSort.java` fixes the increment to `L.length - i` while preserving the recursive splitting and merging design.

```
// Replace the original inversion branch with:  
B[k] = R[j++];  
inv += L.length - i;
```

For example, merging `[2, 3]` and `[1]` produces two cross inversions, `(2, 1)` and `(3, 1)`. The submitted branch counts only one. On `[3, 2, 1]`, the original method reports two inversions instead of three. A sample that happens to pass, such as `[4, 5, 8, 9, 6]`, therefore does not establish correctness.

3.3 Correctness of the corrected method

Each inversion either lies entirely in one half or crosses from the left half to the right half. Recursion counts the first two possibilities. When $R[j] < L[i]$, sortedness implies that $R[j]$ is smaller than all remaining L values, so it creates exactly $L.length - i$ cross inversions. When $L[i] \leq R[j]$, no cross inversion involving that left value remains. Combining recursive and cross counts accounts for every inversion exactly once.

3.4 Return value and counter lifecycle

The original constructor discarded the array returned by `mergesort`. The completed implementation uses `sortArray` to reset `inv`, call `mergesort`, and copy the sorted result back into the caller's array. The public static sort method creates a fresh sorter and returns a long count, ensuring independent counts across runs. The legacy file constructor now validates the requested array size and propagates input errors.

```
int[] sorted = mergesort(array);  
System.arraycopy(sorted, 0, array, 0, array.length);
```

3.5 Complexity

The recurrence $T(n) = 2T(n/2) + O(n)$ gives $O(n \log n)$ time in the best, average, and worst cases. Recursion depth is $O(\log n)$, and peak auxiliary storage is $O(n)$ for this balanced recursive implementation. Repeated allocation adds overhead; a reusable merge buffer could reduce it.

4. Insertion-Sort Methodology

4.1 Counting through shifts

Insertion sort maintains a sorted prefix. It takes the next value as key, shifts larger preceding values one position to the right, and inserts key at the correct location. The supplied InsertionSort.java increments its inversion counter once for every shift. Each shift corresponds to one original pair consisting of a preceding larger value and the current key.

```
while (j >= 0 && arr[j] > key) {  
    arr[j + 1] = arr[j];  
    j = j - 1;  
    inversions++;  
}  
arr[j + 1] = key;
```

4.2 Correctness argument

Before each insertion, the prefix contains the earlier original values in sorted order. Its values greater than key are precisely the earlier values that form inversions with key. The algorithm shifts each of these once and counts one inversion per shift. Summing the shifts over all keys counts every inversion once. The strict comparison `arr[j] > key` avoids counting equal values.

4.3 Worked example

Key	Action	New inversions
5	No value in [4] exceeds 5.	0
8	No value in [4, 5] exceeds 8.	0
9	No value in [4, 5, 8] exceeds 9.	0
6	Shift 9, then 8; insert 6 after 5.	2

The final sequence is [4, 5, 6, 8, 9], and the total is two. The insertionSort method directly changes the array passed by the caller. Its completed main method delegates to the shared driver with insertion sort selected, so it can process supplied files instead of only a hardcoded example. The counter and return type have been widened to long.

4.4 Complexity comparison

Algorithm	Best time	Average time	Worst time
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Fixed-pivot Quicksort	$O(n \log n)$	$O(n \log n)^*$	$O(n^2)$
Insertion sort	$O(n)$	$O(n^2)^*$	$O(n^2)$

*Average bounds assume an appropriate distribution of input order. Insertion sort uses $O(1)$ auxiliary storage, is stable, and can perform well on nearly sorted data. More precisely, its work is $O(n + I)$, where I is the inversion count. On these files, approximately 25 million shifts are required per sequence, making it a useful correctness comparison but a less scalable choice for larger disordered datasets.

5. Experimental Results: Correctness

5.1 Test environment and reference

Experiments ran with OpenJDK 17.0.20. The four completed source files were compiled together using the JDK compiler API. The finished Sorting driver ran all three algorithms on all five supplied files. A separately compiled test harness invoked the actual completed algorithm classes and checked their results independently. Final timing measurements below use these completed classes.

An independent nested-loop reference checked every pair $i < j$ and incremented a long counter when $A[i] > A[j]$. This $O(n^2)$ reference favors transparency over speed. Sorting was independently checked against `Arrays.sort`. Every tested algorithm received a fresh clone of the original input.

5.2 Results for all five source files

File	Quicksort	Merge sort	Insertion sort
source1	25,066,095	25,066,095	25,066,095
source2	24,975,989	24,975,989	24,975,989
source3	25,106,742	25,106,742	25,106,742
source4	25,202,806	25,202,806	25,202,806
source5	25,016,212	25,016,212	25,016,212

Each file contained 10,000 values. All counts above matched the independent pairwise reference, and all resulting arrays matched `Arrays.sort`. The original merge-sort counts were 58,933; 59,274; 59,160; 59,229; and 59,065 for sources 1-5 respectively. They do not measure all inversions and must not be used to rank the sources.

5.3 Edge cases and randomized checks

Each of the three validated methods passed 1,007 additional cases: seven explicit examples and 1,000 random arrays with seed 3364. Random lengths ranged from 0 to 40, and values ranged from -10 to 10, testing negative values and duplicates. Explicit cases covered empty input, one value, ascending order, descending order, [2, 2, 1], the assignment example, and [3, 1, 2].

These checks support correctness on the tested cases; they do not establish robust handling of malformed files, very deep Quicksort recursion, or oversized inputs. Those behaviors remain separate implementation concerns.

6. Performance and Reliability Analysis

6.1 Timing protocol

For each dataset and algorithm, the harness performed three warm-up runs followed by seven measured runs and reported the median. Timing used `System.nanoTime`. Cloning the input happened before the timer started. File parsing, console output, and independent correctness checks were excluded. Array allocations and recursive sorting were included; merge-sort object construction and copying its result back were also included.

These are descriptive local measurements from a single process, not a dedicated Java microbenchmark. Garbage collection, JIT compilation, allocation behavior, and shared-system activity can affect results. No hardware specification or controlled machine isolation was recorded. Small differences should not be treated as universal performance rankings.

File	Quicksort (ms)	Merge sort (ms)	Insertion (ms)
source1	1.999	1.067	8.782
source2	1.955	1.588	8.830
source3	1.759	1.127	9.090
source4	1.114	1.047	9.043
source5	1.135	1.205	9.620

Insertion sort was slower than both recursive methods in every measured file. Merge sort was faster on sources 1-4, and Quicksort was slightly faster on source5 in this run. The narrow input-size range cannot establish asymptotic scaling; the complexity bounds come from algorithm analysis. Merge sort provides the strongest worst-case time guarantee, while Quicksort's fixed pivot remains sensitive to ordering.

6.2 Reliability ranking

Position	Source	Inversions	Fraction of all pairs
1	source2	24,975,989	49.9570%
2	source5	25,016,212	50.0374%
3	source1	25,066,095	50.1372%
4	source3	25,106,742	50.2185%
5	source4	25,202,806	50.4107%

The denominator is $10,000 \times 9,999 / 2 = 49,995,000$ possible pairs. Source2 has 226,817 fewer inversions than source4, a difference of about 0.454 percentage points of all pairs. These similar disagreement levels do not imply that one source is dramatically better. The fractions describe ordering disagreement, not factual credibility or statistical significance.

7. Completed Driver and Integration

7.1 Repairs to the original files

The original `Sorting.java` failed compilation because of a leading byte-order mark, incorrect imports, incorrect `ArrayList` capitalization, missing file imports, an undefined menu variable, an invalid return value from `main`, and an out-of-scope scanner reference. Its dispatch methods were also not connected to the algorithm classes. The completed driver replaces this incomplete flow with a compiling implementation and explicit algorithm selection.

Component	Final behavior
<code>Sorting.java</code>	Reads whitespace-separated integers, reports invalid input with line numbers, supports interactive and command-line use, and dispatches to the three algorithms.
<code>MergeSort.java</code>	Counts every remaining left-side inversion, resets a long counter, and copies the sorted return value back.
<code>InsertionSort.java</code>	Retains shift-based counting and returns long; its main uses the shared file driver.
<code>Quicksort.java</code>	Provides the earlier stable-partition method through a shared sort interface and a file-based main.

7.2 Driver checks

The driver creates a fresh copy for each selected algorithm and compares each sorted result with `Arrays.sort`. Compare mode also requires all three inversion counts to agree. These runtime checks complement the independent pairwise reference used in the experiments. Multiple supplied files are ranked by increasing inversion count; ties have equal reliability even though the output enumerates their positions.

Eight driver tests passed: an invalid algorithm name, a missing algorithm argument, a missing file, the interactive compare option, an invalid interactive selection, an empty file, a malformed token, and whitespace-separated integers with a blank line. Each algorithm-specific main also processed `source1` successfully, for three additional entry-point checks. Missing or invalid input prints an error and exits with status 1 instead of producing a misleading count.

7.3 Compile and run

```
javac *.java
java Sorting --compare source1.txt source2.txt source3.txt \
    source4.txt source5.txt
```

The second command uses shell line continuation; alternatively, enter it on one line without the backslash. Running `java Sorting` without arguments opens the filename and algorithm menu. Use `--algorithm quick`, `merge`, or `insertion` for one method. The `--print` option displays the sorted values; otherwise, the program prints concise results. Filenames are interpreted relative to the working directory unless an absolute path is supplied.

7.4 Remaining limits

The file parser dynamically sizes input, accepts blank lines between values, rejects empty files and invalid or out-of-range integers, and does not impose a 10,000-value limit. Very large files still require memory for input and array copies. Fixed-pivot Quicksort retains its $O(n^2)$ worst case and possible deep-recursion failure; use merge sort for adversarial ordering. Input arrays may be empty at the algorithm-method level even though the driver rejects a file containing no numbers.

8. Conclusions, Future Work, and Team

8.1 Conclusions

Inversion counting turns source-ranking disagreement into an exact pair count. The stable-partition Quicksort method and supplied insertion-sort method correctly count the tested sequences. The completed merge method uses $L.length - i$ and retains its sorted result. All three completed algorithms agreed with the independent reference on the five files and 1,007 additional cases per method. The integrated driver and each individual entry point also passed execution checks.

The reliability order is source2, source5, source1, source3, source4, from fewest to most inversions. Merge sort is the recommended general approach when guaranteed $O(n \log n)$ time is the priority. Insertion sort offers a simple in-place comparison and is especially useful for nearly sorted sequences. The current Quicksort method works on the supplied data but lacks a worst-case $O(n \log n)$ guarantee.

8.2 Future work and submission readiness

The accompanying implementation includes the corrected merge method, long counters, validated input, independent runs, and repaired driver. All five datasets have been checked through the finished application. A reusable merge buffer would be a future optimization to reduce allocation overhead. For Quicksort, changing the pivot requires revisiting the inversion-count proof: simply swapping in a random pivot can change original pair order and invalidate the current counting rule.

A broader performance study should vary input size and order, include ascending, descending, duplicate-heavy, and nearly sorted data, and record the machine and JVM settings. Dedicated microbenchmarking would better separate algorithm costs from JIT and garbage-collection effects. A page-ID alignment stage would make the ranking assumption explicit for real source data.

8.3 Team and contributions

Member	Contribution supplied in team screenshot
Joshua Longoria	Quicksort-based inversion counting (Quicksort.java)
Kylon Ford	Merge-sort-based inversion counting (MergeSort.java)
Mason Womack	Insertion-sort-based inversion counting (InsertionSort.java)
Mia Isabella Troncoso	Report
Marvin	Main function (Sorting.java)

Materials and evidence

Project1.pdf and source1.txt-source5.txt from Project-1 (3).zip; the Quicksort code supplied earlier in this conversation; the attached MergeSort (1).java, InsertionSort.java, and Sorting.java; and the team-contribution screenshot dated October 2, 2026. The completed four-class implementation was derived from these materials. Counts, compiler diagnostics, driver checks, and final timing results were generated locally for this report. No external sources were used.